LAPRAK TEKNOLOGI IOT II

Tugas 4 dan 5

Oleh:

NAMA : M. Kurnia Akbar
NPM : 25782101
KELAS : TRI – 3D

Dosen:

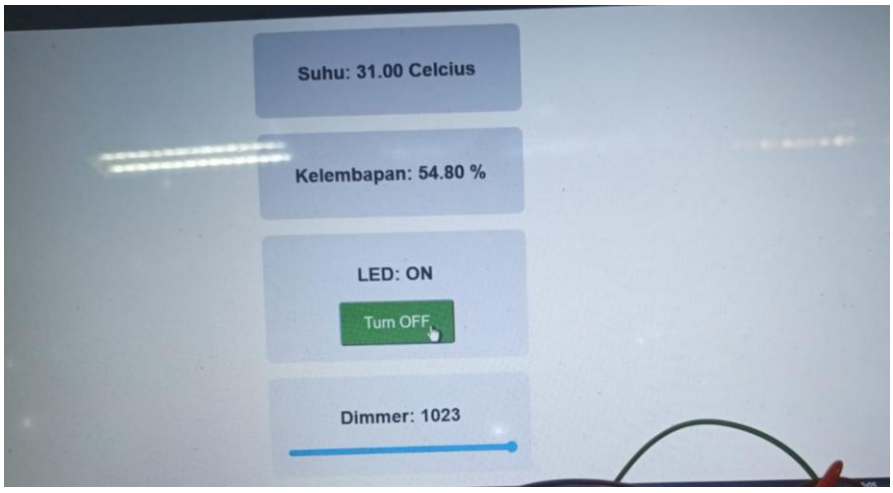
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, S.Kom

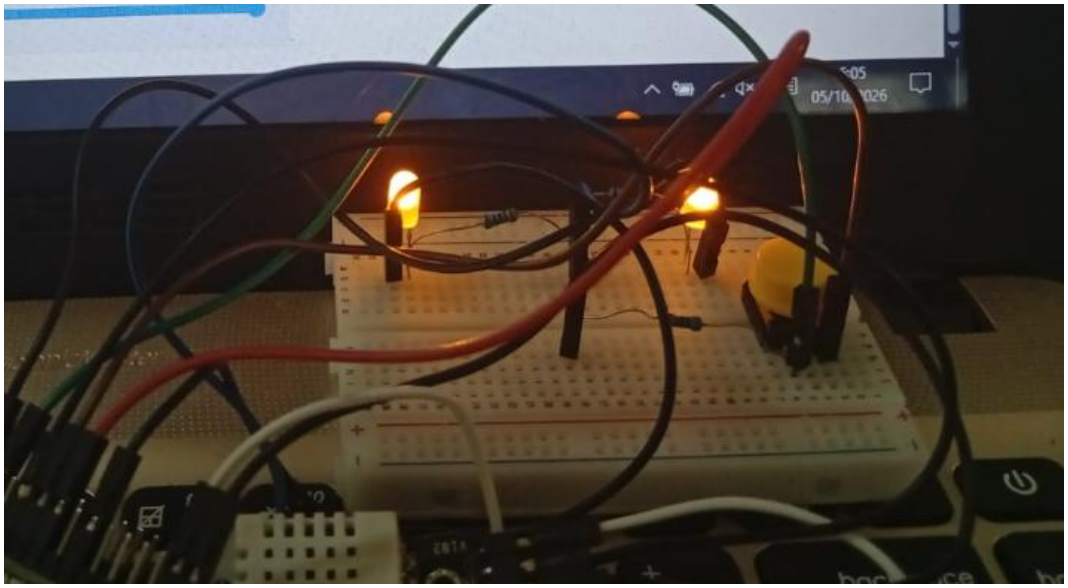
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026

Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analogWrite() (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.





4.7.ino

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Redmi 12";
const char* password = "22345678";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)  -> LED ON/OFF
const byte pwmPin = 5;        // D1 (GPIO 5)   -> LED PWM (BARU)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";
int pwmValue = 0;             // BARU: nilai PWM terakhir (0-1023)
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
    input[type=range] { width: 100%; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>
  <!-- BARU: kartu slider dimmer -->
  <div class="card">
    <h2>Dimmer: <span id="pwmValue">0</span></h2>
    <input type="range" min="0" max="1023" value="0" id="pwmSlider" oninput="sendPWM(this.value)">
  </div>

  <script>
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;
    var dragging = false;      // BARU: true saat slider sedang digeser
    var lastSend = 0;          // BARU: untuk throttle pengiriman

    window.addEventListener('load', onLoad);
    function onLoad(event) {
      initWebSocket();
      var s = document.getElementById('pwmSlider');
      s.addEventListener('pointerdown', function(){ dragging = true; });
      s.addEventListener('pointerup', function(){ dragging = false; });
      s.addEventListener('change', function(){ sendPWM(s.value, true); });  // kirim nilai akhir
```

4.7.ino

```
    }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
      websocket.onopen    = onOpen;
      websocket.onclose   = onClose;
      websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    function toggleLed(){
      websocket.send('toggle');
    }

    // BARU: kirim paket "pwm,nilai" ke NodeMCU (dibatasi tiap 50 ms agar tidak lag)
    function sendPWM(value, force) {
      document.getElementById('pwmValue').innerHTML = value;
      var now = Date.now();
      if ((force || now - lastSend > 50) && websocket.readyState === WebSocket.OPEN) {
        websocket.send('pwm,' + value);
        lastSend = now;
      }
    }

    function onMessage(event) {
      var dataObj = JSON.parse(event.data);

      if(dataObj.suhu !== undefined) {
        document.getElementById('tempValue').innerHTML = dataObj.suhu;
      }

      if(dataObj.hum !== undefined) {
        document.getElementById('humValue').innerHTML = dataObj.hum;
      }

      // BARU: sinkronkan slider (kecuali sedang digeser)
      if(dataObj.pwm !== undefined && !dragging) {
        document.getElementById('pwmSlider').value = dataObj.pwm;
        document.getElementById('pwmValue').innerHTML = dataObj.pwm;
      }

      if(dataObj.led !== undefined) {
        var btn = document.getElementById('toggleBtn');
        var status = document.getElementById('ledStatus');
        if(dataObj.led == "1"){
          status.innerHTML = "ON";
          btn.innerHTML = "Turn OFF";
          btn.className = "btn-on";
        } else {
          status.innerHTML = "OFF";
          btn.innerHTML = "Turn ON";
          btn.className = "btn-off";
        }
      }
    }
  </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// Kirim paket JSON ke SELURUH browser pengunjung
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\", ";
  jsonString += "\"pwm\":\"" + String(pwmValue) + "\"}";   // BARU
  ws.textAll(jsonString);
}

// Handler pesan masuk (dari browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
    data[len] = 0;
    String msg = String((char*)data);
```




```
    if (msg == "toggle") {
      ledState = !ledState;
      notifyClients();
    }
    // BARU: parsing paket "pwm,512"
    else if (msg.startsWith("pwm,")) {
      pwmValue = msg.substring(4).toInt();     // ambil angka setelah koma
      pwmValue = constrain(pwmValue, 0, 1023); // batasi 0-1023
      analogWrite(pwmPin, pwmValue);           // redupkan / terangkan LED
    }
  }
}

// Event handler WebSocket
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients();
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(buttonPin, INPUT);   // resistor pull-down eksternal
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  pinMode(pwmPin, OUTPUT);     // BARU
  analogWrite(pwmPin, 0);      // BARU: awal mati

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

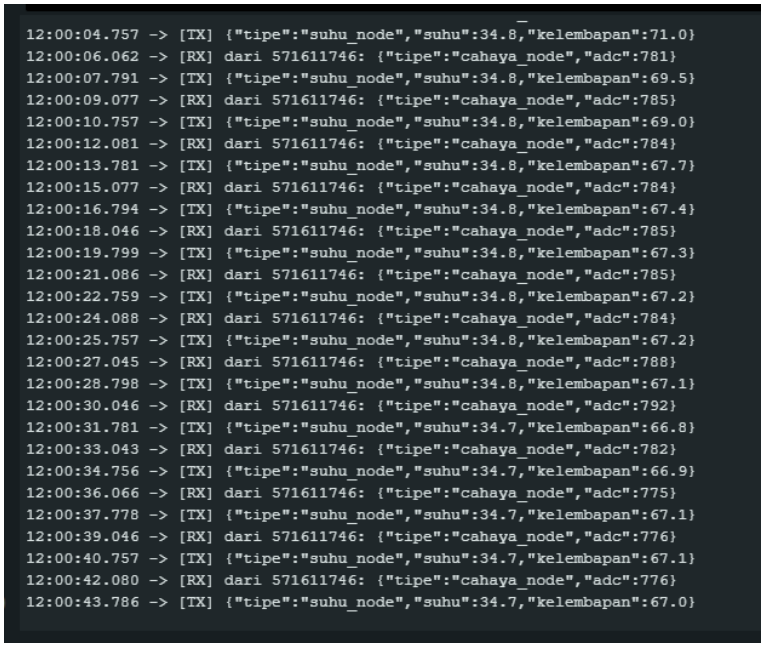
  // 1. Eksekusi LED ON/OFF
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // 2. Baca tombol fisik (debounce)
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients();
      }
    }
  }
```

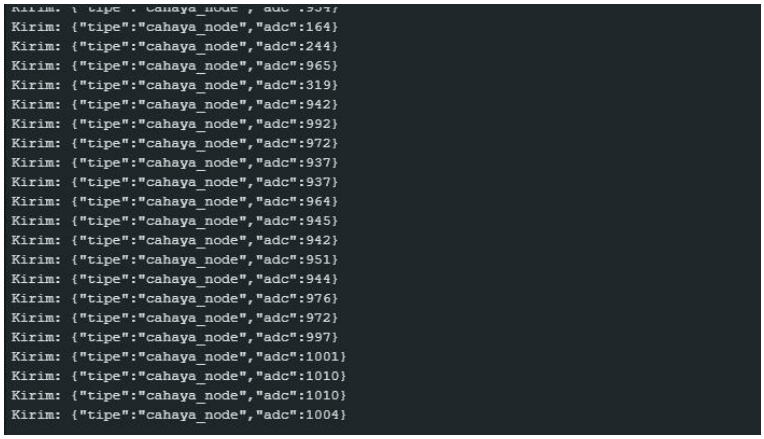
Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh

Kembangkan jaringan mesh pada Praktikum 4 menjadi sistem *monitoring* lingkungan dan kendali otomasi terdistribusi tanpa perantara router. Setiap kelompok membagi peran perangkat sebagai berikut:

Node 1:



Node 2:

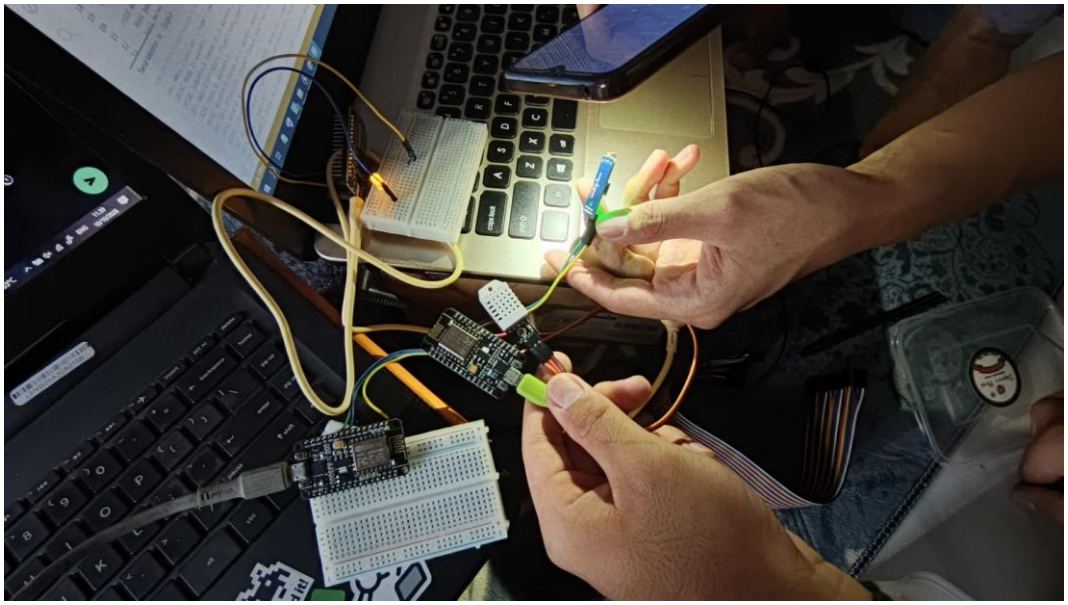


Node 3:

Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
[DATA] Node 2: adc=643
[RULE] suhu=34.0 (batas 35.0) adc=643 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe":"cahaya_node","adc":657}
[DATA] Node 2: adc=657
[RULE] suhu=34.0 (batas 35.0) adc=657 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe":"cahaya_node","adc":674}
[DATA] Node 2: adc=674
[RULE] suhu=34.0 (batas 35.0) adc=674 (batas 300) -> panas=0 gelap=0 -> LED OFF
[RX] dari 571611746: {"tipe":"cahaya_node","adc":282}
[DATA] Node 2: adc=282
[RULE] suhu=34.0 (batas 35.0) adc=282 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe":"cahaya_node","adc":282}
[DATA] Node 2: adc=282
[RULE] suhu=34.0 (batas 35.0) adc=282 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe":"cahaya_node","adc":290}
[DATA] Node 2: adc=290
[RULE] suhu=34.0 (batas 35.0) adc=290 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe":"cahaya_node","adc":269}
[DATA] Node 2: adc=269
[RULE] suhu=34.0 (batas 35.0) adc=269 (batas 300) -> panas=0 gelap=1 -> LED ON
[RX] dari 571611746: {"tipe":"cahaya_node","adc":290}
[DATA] Node 2: adc=290
[RULE] suhu=34.0 (batas 35.0) adc=290 (batas 300) -> panas=0 gelap=1 -> LED ON
```



Pengujian berhasil: ketiga node membentuk jaringan mesh otonom menggunakan painlessMesh tanpa router Wi-Fi eksternal. Node 1 dan Node 2 membaca data riil dari sensor DHT dan LDR setiap 3 detik, lalu menyiarkannya dalam format JSON ke jaringan. Node 3 menerima dan mem-parsing data tersebut dengan ArduinoJson, kemudian Rule Engine-nya menyalakan LED pada pin D6 ketika suhu melebihi 31,0 °C atau nilai ADC cahaya di bawah 300 (kondisi gelap), dan mematikannya ketika kedua parameter kembali normal. Seluruh proses ini terkonfirmasi melalui log Serial Monitor Node 3, sehingga kriteria keberhasilan pengujian terpenuhi.